



UNIVERSIDAD TECNOLÓGICA DEL PERÚ

“Plataforma de comunicaciones unificadas y control de calidad para entornos de call center”

CURSO

Programacion de Interfaces y Dispositivos Perifericos

DOCENTE

Zamudio Ariza, Rene Alejandro

AUTORES

- Gonzales Flores, Gian Piero**
- Loayza Ortega, Jheisson**
- Eddy Jean Pierre Espiritu Soria**

Lima – Perú

1. Resumen ejecutivo

Unified Telecom QA Platform es una plataforma académica de comunicaciones unificadas orientada a un escenario de call center y aseguramiento de calidad. El sistema integra telefonía SIP tradicional, WebRTC desde navegador, backend FastAPI, persistencia PostgreSQL, gestión de identidades con midPoint, servicios de medios con coturn y un reverse proxy Nginx para HTTPS/WSS.

El proyecto se encuentra en un estado avanzado. La documentación interna reporta ejecución satisfactoria de Docker Compose, pruebas backend y frontend, llamadas SIP tradicionales, escenarios WebRTC, auditoría, reportes PDF, métricas de calidad de llamada y análisis con herramientas de calidad y seguridad. Sin embargo, para presentarlo de forma honesta ante un jurado técnico, debe declararse como avance al 90%: la arquitectura base está consolidada y la mejora pendiente prioritaria corresponde a la resiliencia de base de datos, recuperación y failover PostgreSQL.

Dimensión	Estado	Sustento real
Telefonía	Avanzado	Asterisk PJSIP, extensiones 1001/1002 y WebRTC 2001/2002; ConfBridge 700 y video 702.
Backend	Avanzado	FastAPI con rutas de autenticación, llamadas, CDR, presencia, auditoría, métricas, reportes y usuarios.
Base de datos	Avanzado, sin HA	PostgreSQL único; tablas reales users, extensions, call_detail_records, call_sessions, audit_events, quality_evaluations y user_presence.
Frontend	Avanzado	React + SIP.js; pruebas Vitest y Playwright documentadas.
Seguridad	Avanzado con observaciones	JWT, Argon2, RBAC, TLS/WSS/DTLS-SRTP, auditoría; certificados autofirmados de laboratorio.
Calidad	Avanzado	pytest, Vitest, Playwright, k6, SIPp, Trivy y SonarQube según reportes.
Resiliencia	Pendiente	No hay replicación ni failover PostgreSQL; se propone fase adicional.

2. Problema, justificación y objetivo

2.1 Problema

Los entornos de call center requieren controlar la comunicación con agentes, registrar eventos de llamadas, evaluar la calidad de atención, monitorear actividad y generar evidencias auditable. En laboratorios académicos suele existir telefonía aislada, sin trazabilidad suficiente, sin integración con control de calidad y sin una arquitectura clara que permita explicar seguridad, pruebas y operación.

2.2 Justificación

El proyecto aporta una arquitectura integradora que conecta telefonía, backend, frontend, base de datos, identidad y herramientas QA. Esto permite mostrar no solo llamadas, sino también gobierno de usuarios, auditoría, métricas, reportes y controles alineados con ISO/IEC 25010, ISO/IEC 27001 y CMMI de manera académica.

2.3 Objetivo general

Implementar una plataforma de comunicaciones unificadas para entornos de call center que permita autenticación por roles, llamadas SIP/WebRTC, registro CDR, monitoreo, reportes, auditoría y evaluación QA sobre una infraestructura dockerizada.

2.4 Objetivos específicos

- Levantar servicios independientes mediante Docker Compose con redes separadas por función.
- Gestionar autenticación, roles y usuarios desde backend FastAPI con JWT, Argon2 y RBAC.
- Habilitar telefonía SIP tradicional y WebRTC con Asterisk PJSIP, SIP.js y coturn.
- Persistir CDR, sesiones, auditoría y evaluaciones QA en PostgreSQL.
- Exponer reportes, métricas y evidencia de calidad mediante API y panel web.
- Aplicar controles de seguridad y calidad alineados a ISO 27001, ISO 25010 y CMMI.
- Declarar limitaciones reales y planificar mejoras: resiliencia de base de datos, recuperación ante fallos y despliegue productivo.

3. Alcance real del proyecto

Incluido en el alcance	No incluido / pendiente
Login, JWT, Argon2 y roles AgenteCallCenter, Supervisor y AdministradorQA.	Alta disponibilidad productiva y despliegue multinodo.
Voz, video, WSS, ICE, DTLS-SRTP y coturn.	Telefonía PSTN real con operadores públicos.
CDR, grabaciones, métricas WebRTC, MOS y reportes.	Política corporativa de retención/borrado de grabaciones.
midPoint para identidades y sincronización académica.	Certificación formal ISO o evaluación oficial CMMI.
Pruebas automatizadas y herramientas QA.	Failover automático de PostgreSQL.
Operación de comunicaciones, monitoreo, auditoría, reportes y QA en laboratorio.	Resiliencia PostgreSQL avanzada: replicación, failover y recuperación automática.

4. Arquitectura del sistema

La arquitectura se organiza en capas: presentación web, proxy seguro, backend API, persistencia, telefonía, medios, identidad y controles de calidad. Nginx concentra HTTPS/WSS; FastAPI gestiona dominio, seguridad y reportes; Asterisk resuelve señalización y medios; PostgreSQL almacena datos operativos; coturn facilita conectividad WebRTC; midPoint administra identidades; SonarQube participa como herramienta opcional de calidad.

ARQUITECTURA DEL SISTEMA – UNIFIED TELECOM QA PLATFORM

Plataforma académica de comunicaciones unificadas con Asterisk PJSIP, SIP.js, React, FastAPI, PostgreSQL, midPoint, coturn, Nginx y controles QA/ISO.

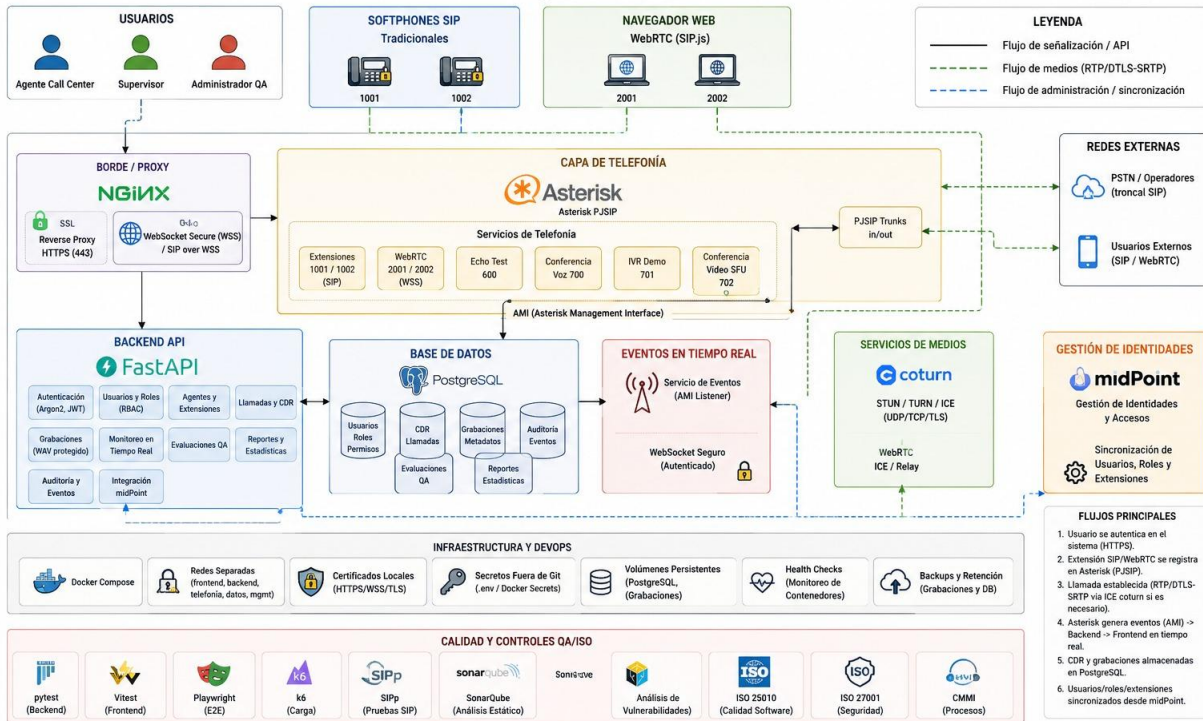


Figura 1. Arquitectura general de Unified Telecom QA Platform.

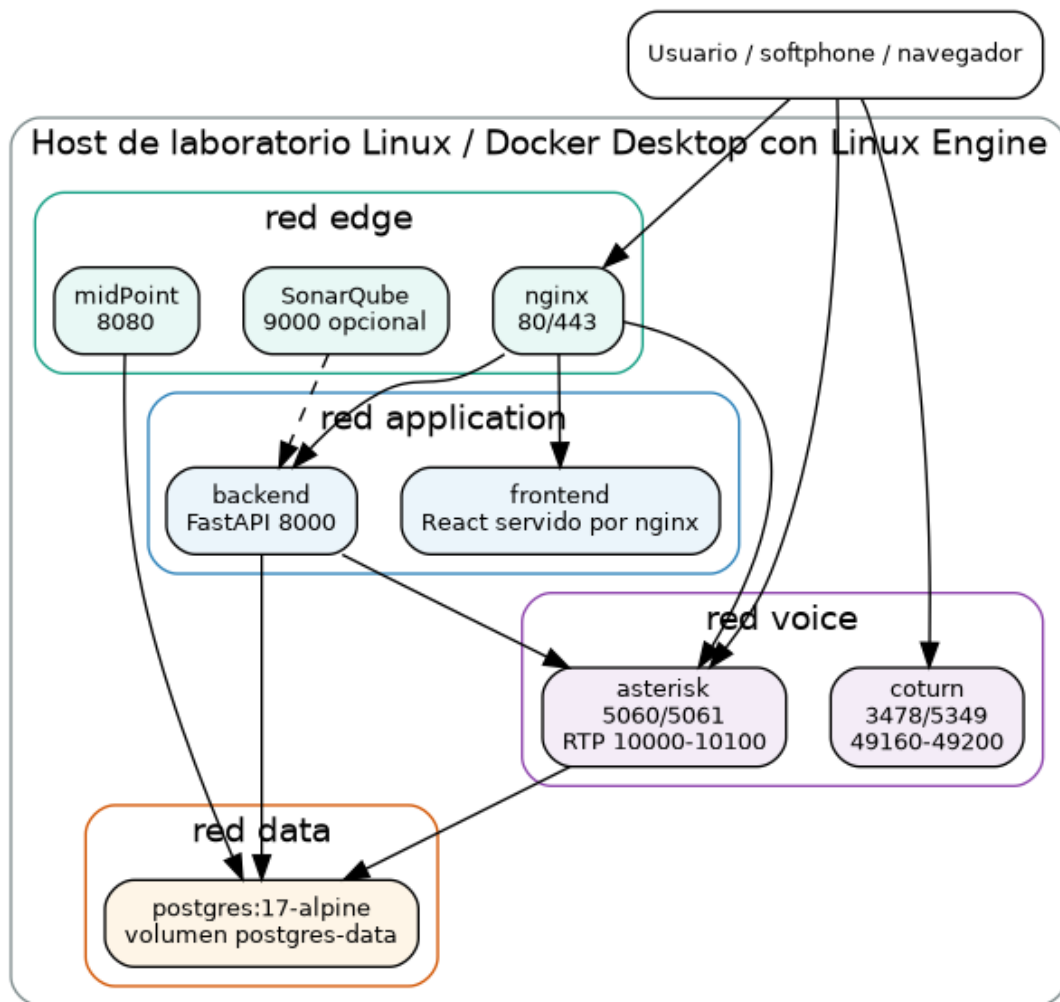


Figura 2. Arquitectura de despliegue sobre Docker Compose y redes separadas.

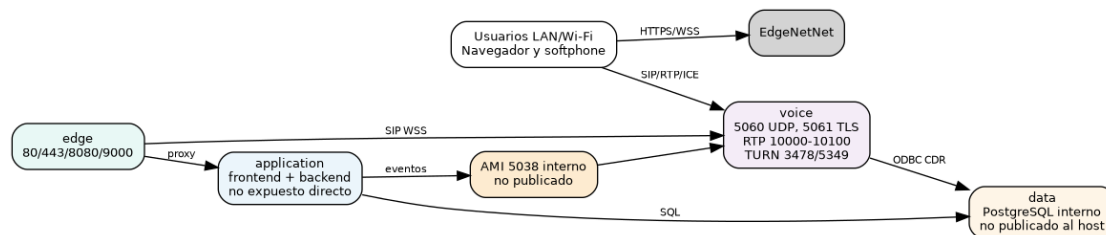


Figura 3. Modelo de red y puertos principales del laboratorio.

5. Infraestructura Docker Compose

El archivo docker-compose.yml define un entorno aislado con el nombre de proyecto unified-telecom-qa-asterik. Este nombre evita interferencias con otros laboratorios Docker del mismo equipo. Los servicios se conectan mediante redes separadas: edge, application, data y voice.

Servicio detectado	Rol técnico
postgres	Base de datos PostgreSQL principal.
backend	API FastAPI.
frontend	Aplicación React servida en contenedor.

asterisk	Central telefónica PJSIP.
coturn	Servidor STUN/TURN para WebRTC.
midpoint-init	Inicialización de repositorio/objetos midPoint.
midpoint	Gestión de identidades IAM.
nginx	Reverse proxy HTTPS/WSS.
sonarqube	Análisis estático opcional de calidad.

docker-compose.yml: servicios base

```

001 name: unified-telecom-qa-asterik
002
003 services:
004   postgres:
005     image: postgres:17-alpine
006     restart: unless-stopped
007     environment:
008       POSTGRES_DB: telecom_qa
009       POSTGRES_USER: telecom_app
010       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?Run scripts/bootstrap.ps1 first}
011       MIDPOINT_DB_PASSWORD: ${MIDPOINT_DB_PASSWORD:?Run scripts/bootstrap.ps1 first}
012     volumes:
013       - postgres-data:/var/lib/postgresql/data
014       - ../infrastructure/postgres/init:/docker-entrypoint-initdb.d:ro
015     networks: [data]
016     healthcheck:
017       test: ["CMD-SHELL", "pg_isready -U telecom_app -d telecom_qa"]
018       interval: 10s
019       timeout: 5s
020       retries: 10
021       start_period: 10s
022     security_opt: ["no-new-privileges:true"]
023
024   backend:
025     build: ./backend
026     restart: unless-stopped
027     environment:
028       APP_ENV: ${APP_ENV:-development}
029       DATABASE_URL: postgresql+asyncpg://telecom_app:${POSTGRES_PASSWORD}@postgres:5432/telecom_qa
030       JWT_SECRET: ${JWT_SECRET:?Run scripts/bootstrap.ps1 first}
031       JWT_ACCESS_MINUTES: ${JWT_ACCESS_MINUTES:-15}
032       APP_ENCRYPTION_KEY: ${APP_ENCRYPTION_KEY:?Run scripts/bootstrap.ps1 first}
033       PROVISIONING_TOKEN: ${PROVISIONING_TOKEN:?Run scripts/bootstrap.ps1 first}
034       DEMO_AGENT_PASSWORD: ${DEMO_AGENT_PASSWORD:?Run scripts/bootstrap.ps1 first}
035       DEMO_AGENT2_PASSWORD: ${DEMO_AGENT2_PASSWORD:?Run scripts/bootstrap.ps1 first}
036       DEMO_SUPERVISOR_PASSWORD: ${DEMO_SUPERVISOR_PASSWORD:?Run scripts/bootstrap.ps1 first}
037       DEMO_ADMIN_PASSWORD: ${DEMO_ADMIN_PASSWORD:?Run scripts/bootstrap.ps1 first}
038       WEBRTC_2001_SECRET: ${WEBRTC_2001_SECRET:?Run scripts/bootstrap.ps1 first}
039       WEBRTC_2002_SECRET: ${WEBRTC_2002_SECRET:?Run scripts/bootstrap.ps1 first}
040       STUN_URL: stun:localhost:${HOST_TURN_PORT:-3478}
041       TURN_URL: turn:localhost:${HOST_TURN_PORT:-3478}
042       TURN_TLS_URL: turns:localhost:${HOST_TURN_TLS_PORT:-5349}
043       TURN_USER: ${TURN_USER:-webrtc}
044       TURN_PASSWORD: ${TURN_PASSWORD:?Run scripts/bootstrap.ps1 first}
045       ASTERISK_HOST: asterisk
046       ASTERISK_HTTP_PORT: 8088
047       ASTERISK_AMI_HOST: asterisk
048       ASTERISK_AMI_PORT: 5038
049       ASTERISK_AMI_USER: backend-events
050       ASTERISK_AMI_SECRET: ${ASTERISK_AMI_SECRET:?Run scripts/bootstrap.ps1 first}
051       RECORDINGS_DIR: /recordings
052     depends_on:
053       postgres: { condition: service_healthy }
054     volumes:
055       - asterisk-recordings:/recordings:ro
056     networks: [application, data, voice]
057     healthcheck:
058       test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8080/health')"]
059       interval: 15s
060       timeout: 5s
061       retries: 10
062     security_opt: ["no-new-privileges:true"]
063
064   frontend:
065     build: ./frontend
066     restart: unless-stopped
067     networks: [application]
068     healthcheck:
069       test: ["CMD-SHELL", "wget -qO- http://127.0.0.1:8080/health || exit 1"]
070     interval: 15s

```

Evidencia de Compose: PostgreSQL, backend y frontend.

docker-compose.yml: voz, IAM y proxy

```
075 asterisk:
076   build:
077     context: ./infrastructure/asterisk
078     args:
079       ASTERISK_VERSION: ${ASTERISK_VERSION:-22.10.0}
080   restart: unless-stopped
081   environment:
082     POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?Run scripts/bootstrap.ps1 first}
083     SIP_1001_SECRET: ${SIP_1001_SECRET:?Run scripts/bootstrap.ps1 first}
084     SIP_1002_SECRET: ${SIP_1002_SECRET:?Run scripts/bootstrap.ps1 first}
085     WEBRTC_2001_SECRET: ${WEBRTC_2001_SECRET:?Run scripts/bootstrap.ps1 first}
086     WEBRTC_2002_SECRET: ${WEBRTC_2002_SECRET:?Run scripts/bootstrap.ps1 first}
087     ASTERISK_AMI_SECRET: ${ASTERISK_AMI_SECRET:?Run scripts/bootstrap.ps1 first}
088     ASTERISK_EXTERNAL_ADDRESS: ${ASTERISK_EXTERNAL_ADDRESS:-}
089     ASTERISK_LOCAL_NET: ${ASTERISK_LOCAL_NET:-}
090     HOST_SIP_UDP_PORT: ${HOST_SIP_UDP_PORT:-5060}
091     HOST_RTP_START: ${HOST_RTP_START:-10000}
092     HOST_RTP_END: ${HOST_RTP_END:-10100}
093   depends_on:
094     postgres: { condition: service_healthy }
095   ports:
096     - "${HOST_SIP_UDP_PORT:-5060}:5060/udp"
097     - "${HOST_SIP_TLS_PORT:-5061}:5061/tcp"
098     - "${HOST_RTP_START:-10000}-${HOST_RTP_END:-10100}:${HOST_RTP_START:-10000}-${HOST_RTP_END:-10100}/udp"
099   volumes:
100     - asterisk-certs:/etc/asterisk/certs
101     - asterisk-recordings:/var/spool/asterisk/monitor
102     - ./infrastructure/asterisk/generated:/etc/asterisk/generated
103   networks: [voice, data]
104   healthcheck:
105     test: ["CMD", "asterisk", "-rx", "core waitfullybooted"]
106     interval: 15s
107     timeout: 8s
108     retries: 12
109     start_period: 30s
110   security_opt: ["no-new-privileges:true"]
111
112 coturn:
113   build: ./infrastructure/coturn
114   restart: unless-stopped
115   environment:
116     TURN_REALM: ${TURN_REALM:-localhost}
117     TURN_USER: ${TURN_USER:-webrtc}
118     TURN_PASSWORD: ${TURN_PASSWORD:?Run scripts/bootstrap.ps1 first}
119   ports:
120     - "${HOST_TURN_PORT:-3478}:3478/tcp"
121     - "${HOST_TURN_PORT:-3478}:3478/udp"
122     - "${HOST_TURN_TLS_PORT:-5349}:5349/tcp"
123     - "${HOST_TURN_TLS_PORT:-5349}:5349/udp"
124     - "${HOST_TURN_RELAY_START:-49160}-${HOST_TURN_RELAY_END:-49200}:49160-49200/udp"
125   volumes:
126     - coturn-certs:/etc/coturn/certs
127   networks: [voice]
128   security_opt: ["no-new-privileges:true"]
129
130 midpoint-init:
131   image: evolveum/midpoint:4.9.1-alpine
132   restart: "no"
133   working_dir: /opt/midpoint
134   entrypoint: ["/bin/sh", "-c"]
135   command:
136     - |
137       if ! /opt/midpoint/bin/ninja.sh -B info; then
138         /opt/midpoint/bin/ninja.sh -B run-sql --create
139         /opt/midpoint/bin/ninja.sh -B run-sql --mode AUDIT --create
140         /opt/midpoint/bin/ninja.sh -B run-sql --scripts /opt/midpoint/objects/drop-superuser.sql
```

Evidencia de Compose: Asterisk, coturn, midPoint, Nginx y redes.

6. Telefonía SIP y WebRTC

El sistema trabaja con dos modalidades. La primera usa softphones SIP tradicionales en extensiones 1001 y 1002. La segunda usa navegador con SIP.js, WebRTC y extensiones 2001/2002 para agentes. Asterisk opera como núcleo de telefonía mediante PJSIP y maneja extensiones, conferencias, IVR, echo test, grabación y CDR.

Extensión	Tipo	Uso
1001	Softphone SIP tradicional	Prueba de llamada desde Zoiper, Linphone o MicroSIP.
1002	Softphone SIP tradicional	Destino SIP de laboratorio.
2001	WebRTC / agente1	Usuario de navegador mediante SIP.js y WSS.
2002	WebRTC / agente2	Usuario de navegador mediante SIP.js y WSS.
600	Echo test	Validación de audio.
700	ConfBridge voz	Conferencia de audio.
701	IVR demo / echo	Demostración de flujo IVR.
702	Video ConfBridge SFU	Videoconferencia WebRTC compatible.

Softphone compatible	Recomendación
Zoiper	Útil para pruebas rápidas en PC o celular; revisar transporte SIP y códecs.
Linphone	Opción abierta y frecuente en laboratorios SIP.
MicroSIP	Ligero en Windows para extensiones SIP tradicionales.
Navegador Chromium/Chrome/Edge	Para WebRTC mediante React + SIP.js, WSS e ICE.


```

sip.ts: cliente WebRTC SIP.js
020   rtt_ms: number | null;
021   bitrate_kbps: number | null;
022 }
023
024 export interface SipCallbacks {
025   onRegistration: (state: RegistrationState) => void;
026   onSession: (session: Session | null) => void;
027   onIncoming: (call: IncomingCall | null) => void;
028   onError: (message: string) => void;
029 }
030
031 export class SipClient {
032   private userAgent?: UserAgent;
033   private registerer?: Registerer;
034   private session?: Session;
035   private pendingInvitation?: Invitation;
036   private doNotDisturb = false;
037   private sessionStartedAt?: number;
038
039   constructor(private readonly callbacks: SipCallbacks) {}
040
041   async connect(config: SipConfig): Promise<void> {
042     this.callbacks.onRegistration("connecting");
043     const uri = UserAgent.makeURI(`sip:${config.extension}@${config.sip_domain}`);
044     if (!uri) throw new Error("URI SIP invalida");
045     const websocket = new URL(config.websocket_url);
046     websocket.host = window.location.host;
047     this.userAgent = new UserAgent({
048       uri,
049       authorizationUsername: config.authorization_username,
050       authorizationPassword: config.password,
051       transportOptions: { server: websocket.toString() },
052       sessionDescriptionHandlerFactoryOptions: {
053         peerConnectionConfiguration: { iceServers: config.ice_servers },
054       },
055       delegate: {
056         onInvite: async (invitation: Invitation) => {
057           if (this.doNotDisturb) {
058             await invitation.reject({ statusCode: 486, reasonPhrase: "Do Not Disturb" });
059             this.callbacks.onError("Llamada rechazada por No Molestar");
060             return;
061           }
062           const withVideo = invitation.request.body?.includes("m=video") ?? false;
063           this.pendingInvitation = invitation;
064           this.callbacks.onIncoming({
065             from: invitation.remoteIdentity.uri.user ?? "desconocido",
066             displayName: invitation.remoteIdentity.displayName || "Llamada entrante",
067             withVideo,
068           });
069         },
070       },
071     });
072     this.registerer = new Registerer(this.userAgent);
073     this.registerer.stateChange.addListener((state) => {
074       if (state === RegistererState.Registered) this.callbacks.onRegistration("registered");
075       if (state === RegistererState.Unregistered) this.callbacks.onRegistration("offline");
076       if (state === RegistererState.Terminated) this.callbacks.onRegistration("error");
077     });
078     try {
079       await this.userAgent.start();
080       await this.registerer.register();
081     } catch (error) {
082       this.callbacks.onRegistration("error");
083       throw error;
084     }
085   }
086
087   async call(destination: string, withVideo: boolean): Promise<Session> {
088     if (!this.userAgent) throw new Error("SIP no registrado");
089     const target = UserAgent.makeURI(`sip:${destination}@localhost`);
090     if (!target) throw new Error("Destino invalido");
091     const inviter = new Inviter(this.userAgent, target, {
092       sessionDescriptionHandlerOptions: {
093         constraints: { audio: true, video: withVideo },
094       },
095     });
096     this.bindSession(inviter);
097     await inviter.invite();
098     return inviter;
099   }
100
101   async acceptIncoming(): Promise<Session> {
102     if (!this.pendingInvitation) throw new Error("No hay llamada entrante");
103     const invitation = this.pendingInvitation;
104     const withVideo = invitation.request.body?.includes("m=video") ?? false;
105     this.pendingInvitation = undefined;
106     this.callbacks.onIncoming(null);
107     this.bindSession(invitation);
108     await invitation.accept({
109       sessionDescriptionHandlerOptions: { constraints: { audio: true, video: withVideo } },
110     });
111     return invitation;
112   }
113
114   async rejectIncoming(): Promise<void> {
115     if (!this.pendingInvitation) return;
116     const invitation = this.pendingInvitation;
117     this.pendingInvitation = undefined;
118     this.callbacks.onIncoming(null);
119     await invitation.reject({ statusCode: 486, reasonPhrase: "Busy Here" });
120   }

```

Evidencia de frontend: registro WebRTC y construcción de sesiones SIP.js.

extensions.conf: dialplan Asterisk

```
001 [general]
002 static=yes
003 writeprotect=yes
004 clearglobalvars=no
005
006 [globals]
007 CONSOLE=Console/default
008
009 [internal]
010 exten => _[12]XXX,1,NoOp(Internal call ${CALLERID(num)} to ${EXTEN})
011     same => n,Set(CDR(userfield)=media=${IF(${CHANNEL(pjsip,secure)}="1"?secure:legacy)})
012     same => n,Set(RECORDING_FILE=${UNIQUEID}.wav)
013     same => n,MixMonitor(/var/spool/asterisk/monitor/${RECORDING_FILE},b)
014     same => n,Dial(PJSIP/${EXTEN},30)
015     same => n,StopMixMonitor()
016     same => n,Hangup()
017
018 exten => 600,1,Answer()
019     same => n,Echo()
020     same => n,Hangup()
021
022 exten => 700,1,Answer()
023     same => n,Set(CDR(userfield)=media=conference)
024     same => n,ConfBridge(telecom-qa,default_bridge,default_user)
025     same => n,Hangup()
026
027 exten => 701,1,Answer()
028     same => n,Playback(demo-congrats)
029     same => n,Echo()
030     same => n,Hangup()
031
032 exten => 702,1,Answer()
033     same => n,Set(CDR(userfield)=media=video-conference)
034     same => n,ConfBridge(telecom-video,video_bridge,video_user)
035     same => n,Hangup()
```

Evidencia de telefonía: extensiones, echo, conferencia y tratamiento de llamadas.

7. Casos de uso

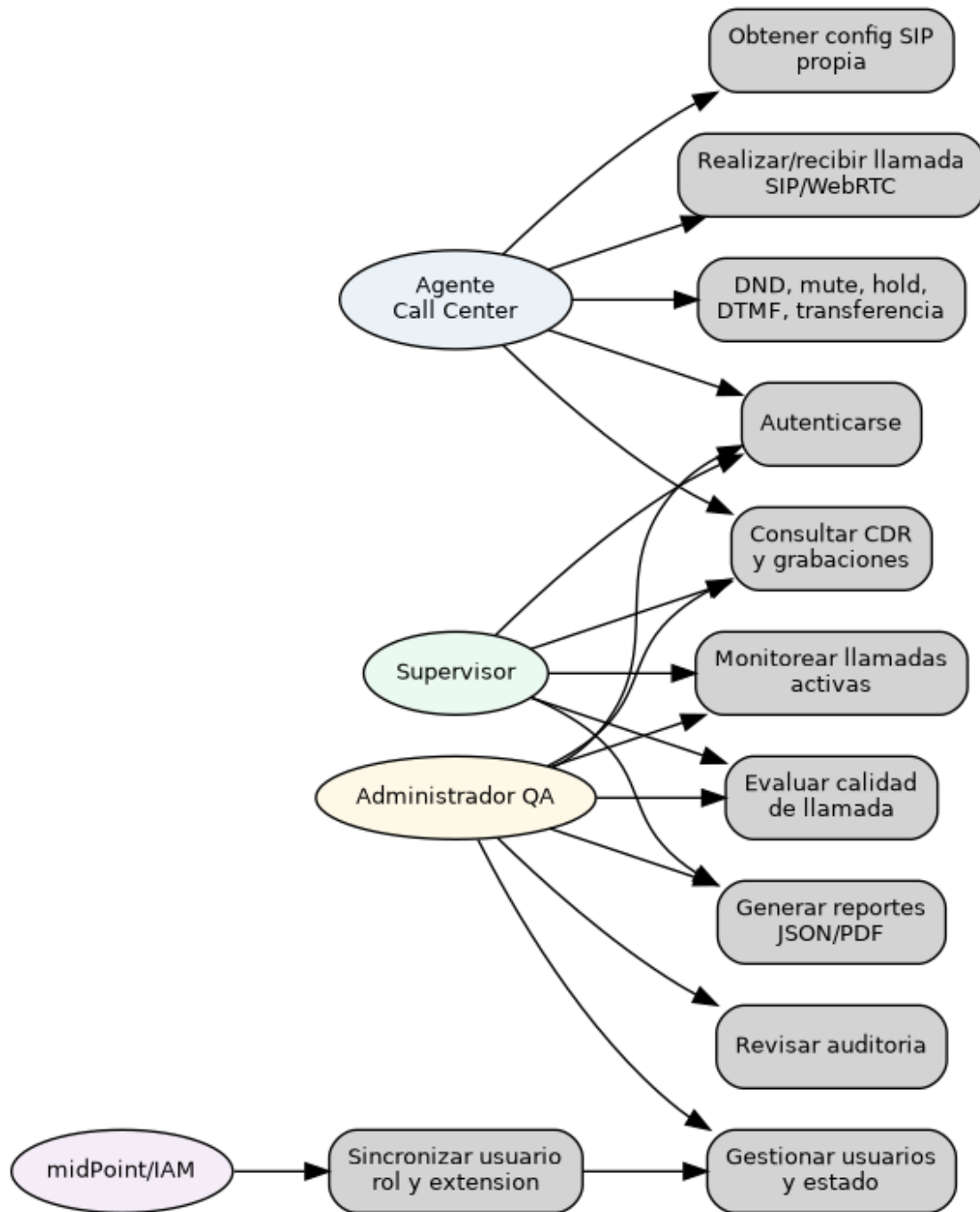


Figura 4. Diagrama de casos de uso reconstruido desde roles y endpoints reales.

Los casos de uso fueron reconstruidos a partir de los roles implementados y de los endpoints reales definidos en el backend FastAPI.

CU	Actor	Caso de uso	Descripción	Estado
CU-01	Agente/Supervisor/Admin	Autenticarse	Ingreso con usuario, contraseña, JWT corto y rol asociado.	Implementado
CU-02	Agente	Obtener configuración SIP	Consulta de extensión propia y secreto	Implementado

			protegido para registro WebRTC.	
CU-03	Agente	Realizar llamada	Uso de SIP.js/Asterisk para comunicación con otra extensión.	Implementado
CU-04	Agente	Controlar llamada	DND, mute, hold, DTMF, transferencia y conferencia según interfaz/telefonía.	Implementado según QA
CU-05	Supervisor/Admin	Consultar CDR	Listado, filtros y paginación de llamadas.	Implementado
CU-06	Supervisor/Admin	Evaluar calidad	Registro de evaluación QA sobre llamada.	Implementado
CU-07	Supervisor/Admin	Generar reporte	Resumen y PDF con auditoría de descarga.	Implementado
CU-08	Admin QA	Revisar auditoría	Consulta de eventos: login, acciones, llamadas, provisioning.	Implementado
CU-09	Admin QA / midPoint	Sincronizar usuarios	Alta/cambio/baja idempotente hacia backend/Asterisk.	Implementado

8. Requisitos y trazabilidad

Los requisitos provienen del SRS, historias de usuario y matriz de trazabilidad presentes en la carpeta docs. Para reforzar el documento ante jurado, se consolida una matriz requisito → implementación → evidencia.

ID	Requisito	Archivo/módulo	Evidencia técnica	Estado
RF-01	Autenticación segura	backend/app/routers/auth.py	POST /api/auth/login; Argon2 + JWT	Implementado
RF-02	Consulta del usuario actual	backend/app/routers/auth.py	GET /api/auth/me	Implementado
RF-03	Configuración SIP propia	backend/app/routers/calls.py	GET /api/extensions/me/sip-config	Implementado
RF-04	Listado y paginación CDR	backend/app/routers/calls.py	GET /api/calls y /api/calls/page	Implementado
RF-05	Eventos de llamada	backend/app/routers/calls.py	POST /api/calls/events	Implementado
RF-06	Calidad WebRTC/MOS	backend/app/routers/calls.py	POST /api/calls/quality	Implementado
RF-07	Presencia/DND	backend/app/routers/calls.py	GET/POST /api/presence	Implementado
RF-08	Grabaciones protegidas	backend/app/routers/calls.py	GET /api/recordings/{uniqueid}	Implementado
RF-09	Auditoría	backend/app/routers/governance.py	GET /api/audit	Implementado
RF-10	Reportes y PDF	backend/app/routers/governance.py	GET /api/reports/summary y .pdf	Implementado
RF-11	Usuarios y provisioning	backend/app/routers/governance.py	GET/PATCH users; PUT provisioning	Implementado
RF-12	Eventos en tiempo real	backend/app/routers/events.py	WebSocket /api/events/ws	Implementado
RF-13	Resiliencia BD	No existe configuración real	Sin PostgreSQL réplica/failover	Pendiente

9. Inventario real de API

Archivo	Método	Ruta	Función
auth.py	POST	/login	login
auth.py	GET	/me	me
calls.py	GET	/extensions/me/sip-config	sip_config
calls.py	GET	/calls	list_calls

calls.py	GET	/calls/page	call_page
calls.py	POST	/calls/events	call_event
calls.py	POST	/calls/quality	call_quality
calls.py	GET	/presence	current_presence
calls.py	POST	/presence	presence
calls.py	GET	/recordings/{uniqueid}	recording
governance.py	GET	/audit	audit_events
governance.py	GET	/metrics/quality	quality_metrics
governance.py	GET	/metrics/quality-summary	quality_summary
governance.py	GET	/monitoring/active-calls	active_calls
governance.py	GET	/evaluations	evaluations
governance.py	POST	/evaluations	create_evaluation
governance.py	GET	/reports/summary	report_summary
governance.py	GET	/reports/summary.pdf	report_summary_pdf
governance.py	GET	/users	users
governance.py	PATCH	/users/{username}/status	update_user_status
governance.py	GET	/services/status	services_status
governance.py	PUT	/provisioning/users/{username}	provision_user
events.py	WS	/api/events/ws	realtime_events

10. Modelo de base de datos

El modelo real se identifica en backend/app/models.py mediante SQLAlchemy. La base de datos soporta usuarios, extensiones, presencia, CDR, sesiones de llamada, auditoría y evaluaciones QA, por lo que cubre la operación técnica de comunicaciones, monitoreo y calidad del laboratorio.

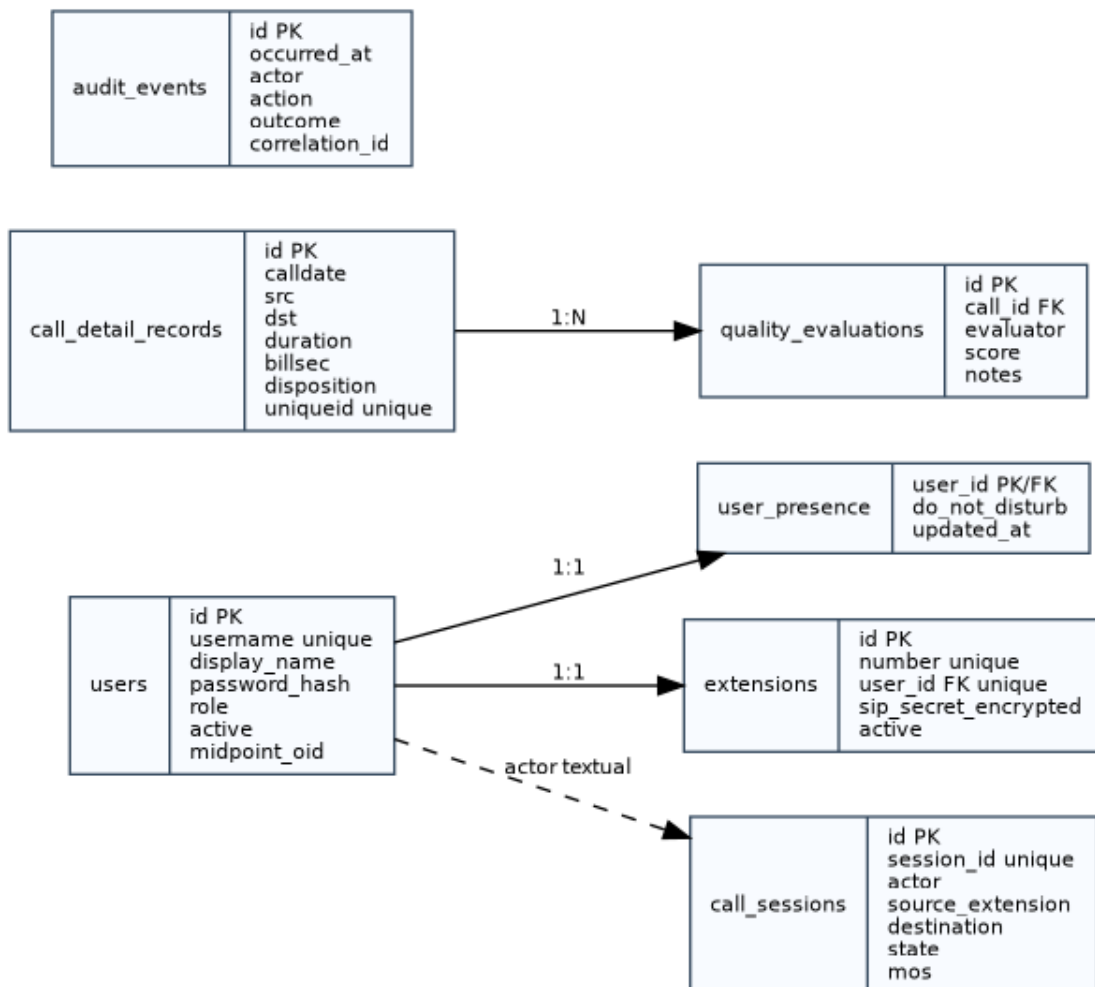


Figura 5. Modelo entidad-relación reconstruido desde backend/app/models.py.

Tabla 2. Resumen funcional de entidades persistentes.

Tabla	Propósito dentro del sistema
users	Usuarios, credenciales hasheadas, roles y vínculo con midPoint.
user_presence	Presencia operativa del usuario y estado DND.
extensions	Extensiones SIP/WebRTC asociadas a usuarios y secretos SIP cifrados.
call_detail_records	CDR de Asterisk: origen, destino, duración, estado e identificadores de llamada.
call_sessions	Sesiones activas/históricas y métricas WebRTC como jitter, RTT, pérdida y MOS.
audit_events	Trazabilidad de acciones: actor, acción, resultado, IP y correlación.
quality_evaluations	Evaluaciones QA asociadas a llamadas registradas en CDR.

10.1 Tabla users

Campo	Tipo real	Restricción u observación
id	INTEGER	PK, autoincremental
username	VARCHAR(80)	UNIQUE, INDEX, obligatorio
display_name	VARCHAR(160)	obligatorio
password_hash	TEXT	obligatorio; almacena hash Argon2
role	VARCHAR(40)	INDEX; rol RBAC
active	BOOLEAN	DEFAULT TRUE
midpoint_oid	VARCHAR(80)	UNIQUE, nullable; identificador midPoint
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT utc_now
updated_at	TIMESTAMP WITH TIME ZONE	DEFAULT utc_now, ON UPDATE utc_now

10.2 Tabla user_presence

Campo	Tipo real	Restricción u observación
user_id	INTEGER	PK, FK users.id
do_not_disturb	BOOLEAN	DEFAULT FALSE; estado DND
updated_at	TIMESTAMP WITH TIME ZONE	DEFAULT utc_now, ON UPDATE utc_now

10.3 Tabla extensions

Campo	Tipo real	Restricción u observación
id	INTEGER	PK, autoincremental
number	VARCHAR(16)	UNIQUE, INDEX; extensión SIP/WebRTC
user_id	INTEGER	FK users.id, UNIQUE
sip_secret_encrypted	TEXT	secreto SIP cifrado
active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT utc_now

10.4 Tabla call_detail_records

Campo	Tipo real	Restricción u observación
id	INTEGER	PK, autoincremental
calldate	TIMESTAMP WITH TIME ZONE	INDEX; fecha de llamada
clid	VARCHAR(128)	nullable
src	VARCHAR(80)	INDEX, nullable; origen
dst	VARCHAR(80)	INDEX, nullable; destino
dcontext	VARCHAR(80)	nullable; contexto dialplan
channel	VARCHAR(160)	nullable
dstchannel	VARCHAR(160)	nullable
lastapp	VARCHAR(80)	nullable
lastdata	VARCHAR(160)	nullable
duration	INTEGER	DEFAULT 0; duración total
billsec	INTEGER	DEFAULT 0; tiempo facturable/contestada
disposition	VARCHAR(45)	INDEX, nullable; ANSWERED/NO ANSWER/etc.
amaflags	INTEGER	DEFAULT 0
accountcode	VARCHAR(80)	nullable
uniqueid	VARCHAR(150)	UNIQUE; identificador Asterisk
userfield	VARCHAR(255)	nullable
peeraccount	VARCHAR(80)	nullable
linkedid	VARCHAR(150)	INDEX, nullable
sequence	INTEGER	nullable

10.5 Tabla audit_events

Campo	Tipo real	Restricción u observación
id	INTEGER	PK, autoincremental
occurred_at	TIMESTAMP WITH TIME ZONE	DEFAULT utc_now, INDEX
actor	VARCHAR(80)	INDEX; usuario/servicio que ejecuta la acción
action	VARCHAR(80)	INDEX; acción auditada
outcome	VARCHAR(24)	INDEX; resultado
source_ip	VARCHAR(64)	nullable
correlation_id	VARCHAR(64)	INDEX
details	JSON	DEFAULT dict; metadatos del evento

10.6 Tabla call_sessions

Campo	Tipo real	Restricción u observación
id	INTEGER	PK, autoincremental
session_id	VARCHAR(128)	UNIQUE, INDEX
actor	VARCHAR(80)	INDEX
source_extension	VARCHAR(16)	INDEX
destination	VARCHAR(16)	INDEX
media	VARCHAR(12)	DEFAULT audio
state	VARCHAR(24)	DEFAULT started, INDEX
started_at	TIMESTAMP WITH TIME ZONE	DEFAULT utc_now, INDEX
answered_at	TIMESTAMP WITH TIME ZONE	nullable
ended_at	TIMESTAMP WITH TIME ZONE	nullable
held	BOOLEAN	DEFAULT FALSE
transferred_to	VARCHAR(16)	nullable
packets_received	INTEGER	DEFAULT 0
packets_lost	INTEGER	DEFAULT 0
jitter_ms	FLOAT	nullable
rtt_ms	FLOAT	nullable
bitrate_kbps	FLOAT	nullable
mos	FLOAT	nullable; calidad estimada

10.7 Tabla quality_evaluations

Campo	Tipo real	Restricción u observación
id	INTEGER	PK, autoincremental
call_id	INTEGER	FK call_detail_records.id, INDEX
evaluator	VARCHAR(80)	INDEX
score	INTEGER	puntuación QA
notes	TEXT	DEFAULT vacío
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT utc_now, INDEX

Nota de validación: los tipos y restricciones anteriores fueron corregidos a partir de backend/app/models.py. Se eliminaron las descripciones genéricas generadas automáticamente para evitar ambigüedad técnica ante el jurado.

11. Seguridad e ISO/IEC 27001

El proyecto incorpora seguridad desde la autenticación, la comunicación y la operación. Usa Argon2 para hashes de contraseña, JWT para sesiones, RBAC por rol, HTTPS/WSS, DTLS-SRTP para WebRTC, secretos fuera de Git, auditoría y redes Docker separadas. Aun así, la documentación declara que los certificados son autofirmados y exclusivos de laboratorio.

Control	Descripción	Evidencia
Control de acceso	Roles AgenteCallCenter, Supervisor, AdministradorQA.	JWT + dependencias FastAPI.
Protección de credenciales	Argon2 para contraseñas; secretos SIP cifrados.	backend/app/security.py y modelos.
Comunicación segura	HTTPS, WSS, DTLS-SRTP y TURN TLS.	Nginx, Asterisk, coturn.
Auditoría	Eventos con actor, acción, resultado, IP y correlation ID.	audit_events.
Segmentación	Redes Docker edge/application/data/voice.	docker-compose.yml.
Gestión de secretos	Variables .env generadas por bootstrap; no capturar ni versionar.	README y scripts.
Pendiente productivo	PKI corporativa, cifrado en reposo de grabaciones, retención/borrado, HA.	Declarado como observación.

12. Calidad e ISO/IEC 25010

La documentación del proyecto contiene matrices ISO 25010, ISO 27001, CMMI, plan de pruebas, casos de prueba y reporte QA. El reporte final indica aprobación para demostración académica con observaciones.

Característica ISO 25010	Aplicación en el proyecto	Resultado/evidencia
Adecuación funcional	Login, roles, llamadas, CDR, QA, reportes y auditoría.	Validado por pruebas API/frontend.
Eficiencia de desempeño	k6: 300/300 peticiones, 0% error, p95 2.34 ms.	Reporte QA final.
Compatibilidad	SIP tradicional + WebRTC; Chrome/Edge y softphone seleccionado.	Plan de pruebas y QA.
Usabilidad	Flujo login-llamada y dashboard de agente.	Frontend React.
Fiabilidad	Health checks y evidencias de servicios activos.	Compose + acta final.
Seguridad	Trivy 0 vulnerabilidades y SonarQube 0 bugs/vulnerabilidades.	Reporte QA.
Mantenibilidad	Cobertura backend 82.16%, frontend 95.05% líneas; 9 code smells no bloqueantes.	Reporte QA.
Portabilidad	Arranque documentado Windows/Linux/macOS mediante Docker Compose.	README y scripts bootstrap.

13. Pruebas, resultados reportados y herramientas QA

Tabla consolidada del reporte QA final.

Gate	Resultado observado	Estado
Docker Compose	docker compose up -d --build finalizo correctamente	Aprobado

Servicios	PostgreSQL, backend, frontend, Asterisk, coturn, midPoint, Nginx y SonarQube activos	Aprobado
Backend	pytest: 11/11; cobertura 82.16%; API, RBAC, PDF, eventos y aprovisionamiento	Aprobado
Frontend	ESLint/build; Vitest 30/30; líneas 95.05%; ramas 80.41%	Aprobado
SIP tradicional	1001-1002: Digest, INVITE/ACK/BYE, RTP PCMU bidireccional, CDR 55	Aprobado
WebRTC	Playwright: WSS, voz, video 2001/2002 y dos agentes en SFU 702	Aprobado
Telefonia avanzada	Hold/resume, DTMF INFO, ConfBridge 700 y videoconferencia 702	Aprobado
Tiempo real	AMI interno conectado y WebSocket autenticado por Nginx con HTTP 101	Aprobado
Reporte PDF	PDF HTTPS renderizado, métricas legibles y exportación auditada	Aprobado
CDR	Dos registros finales ANSWERED, media=secure	Aprobado
Grabaciones	WAV MixMonitor persistido y descarga FastAPI con RBAC	Aprobado
Calidad de llamada	Jitter, RTT, perdida y MOS 4.49/4.50 persistidos	Aprobado
IAM	agente3, rol AgenteCallCenter, extension 2003 y PJSIP recargado	Aprobado
Auditoria	Login, lectura SIP/CDR, inicio/fin y provisionamiento persistidos	Aprobado
Carga	k6: 300/300, 0% error, p95 2.34 ms	Aprobado
Trivy	0 vulnerabilidades, 0 secretos, 0 misconfiguraciones	Aprobado
SonarQube	Gate OK; cobertura 85.3%; 0 bugs, 0 vulnerabilidades, 9 code smells y 0 hotspots	Aprobado
npm audit	0 vulnerabilidades	Aprobado

Evidencia ejecutada según acta final.

Evidencia	Resultado
Arranque	docker compose up -d --build exitoso
Salud	8 servicios activos; healthchecks disponibles aprobados
PJSIP/ODBC	5 endpoints y conexion ODBC activa
WebRTC	Playwright Chromium aprobo voz/video 2001-2002 y video remoto SFU en 702
Controles SIP	Retener/reanudar, DTMF INFO, conferencia 700 y videoconferencia 702 aprobados
Tiempo real	Cuenta AMI interna conectada y WebSocket autenticado devolvio HTTP 101
PDF	Exportacion HTTPS %PDF, 2,603 bytes, render visual sin defectos
CDR	ANSWERED, 5 s, origen 2001, destino 2002, media=secure
Grabacion	MixMonitor genero WAV; descarga autorizada HTTP 200 y 24,044 bytes
Calidad	WebRTC getStats() persistio MOS 4.49/4.50, jitter, RTT y perdida
Servicios	API, PostgreSQL, PBX, grabacion, IVR y red reportaron ok
Auditoria	LOGIN, SIP_CONFIG_READ, CALL_STARTED/ENDED y provisionamiento
midPoint	Objetos idempotentes y agente3 -> 2003 sincronizado
Carga	300 peticiones, 0% error, p95 2.34 ms
Frontend	30/30; líneas 95.05%; ramas 80.41%; build y ESLint aprobados
Backend	11/11; cobertura 82.16%; filtros, MOS, WebSocket, PDF y evaluacion QA
npm audit	0 vulnerabilidades
Trivy	0 vulnerabilidades, 0 secretos, 0 misconfiguraciones
SonarQube	Gate OK, cobertura 85.3%, 0 bugs/vulnerabilidades, 9 code smells, 0 hotspots

14. Evidencias de código relevante

```
models.py: modelo de datos real

015 class User(Base):
016     __tablename__ = "users"
017
018     id: Mapped[int] = mapped_column(primary_key=True)
019     username: Mapped[str] = mapped_column(String(80), unique=True, index=True)
020     display_name: Mapped[str] = mapped_column(String(160))
021     password_hash: Mapped[str] = mapped_column(Text)
022     role: Mapped[str] = mapped_column(String(40), index=True)
023     active: Mapped[bool] = mapped_column(Boolean, default=True)
024     midpoint_oid: Mapped[str | None] = mapped_column(String(80), unique=True, nullable=True)
025     created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utc_now)
026     updated_at: Mapped[datetime] = mapped_column(
027         DateTime(timezone=True), default=utc_now, onupdate=utc_now
028     )
029
030     extension: Mapped["Extension | None"] = relationship(back_populates="user", uselist=False)
031
032
033 class UserPresence(Base):
034     __tablename__ = "user_presence"
035
036     user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), primary_key=True)
037     do_not_disturb: Mapped[bool] = mapped_column(Boolean, default=False)
038     updated_at: Mapped[datetime] = mapped_column(
039         DateTime(timezone=True), default=utc_now, onupdate=utc_now
040     )
041
042
043 class Extension(Base):
044     __tablename__ = "extensions"
045
046     id: Mapped[int] = mapped_column(primary_key=True)
047     number: Mapped[str] = mapped_column(String(16), unique=True, index=True)
048     user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), unique=True)
049     sip_secret_encrypted: Mapped[str] = mapped_column(Text)
050     active: Mapped[bool] = mapped_column(Boolean, default=True)
051     created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utc_now)
052
053     user: Mapped[User] = relationship(back_populates="extension")
054
055
056 class CallDetailRecord(Base):
057     __tablename__ = "call_detail_records"
058
059     id: Mapped[int] = mapped_column(primary_key=True)
060     calldate: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)
061     clid: Mapped[str | None] = mapped_column(String(128), nullable=True)
062     src: Mapped[str | None] = mapped_column(String(80), index=True, nullable=True)
063     dst: Mapped[str | None] = mapped_column(String(80), index=True, nullable=True)
064     dcontext: Mapped[str | None] = mapped_column(String(80), nullable=True)
065     channel: Mapped[str | None] = mapped_column(String(160), nullable=True)
066     dstchannel: Mapped[str | None] = mapped_column(String(160), nullable=True)
067     lastapp: Mapped[str | None] = mapped_column(String(80), nullable=True)
068     lastdata: Mapped[str | None] = mapped_column(String(160), nullable=True)
069     duration: Mapped[int] = mapped_column(Integer, default=0)
070     billsec: Mapped[int] = mapped_column(Integer, default=0)
071     disposition: Mapped[str | None] = mapped_column(String(45), index=True, nullable=True)
072     amflags: Mapped[int] = mapped_column(Integer, default=0)
073     accountcode: Mapped[str | None] = mapped_column(String(80), nullable=True)
074     uniqueid: Mapped[str] = mapped_column(String(150), unique=True)
075     userfield: Mapped[str | None] = mapped_column(String(255), nullable=True)
076     peeraccount: Mapped[str | None] = mapped_column(String(80), nullable=True)
077     linkedid: Mapped[str | None] = mapped_column(String(150), index=True, nullable=True)
078     sequence: Mapped[int | None] = mapped_column(Integer, nullable=True)
079
080
081 class AuditEvent(Base):
082     __tablename__ = "audit_events"
083
084     id: Mapped[int] = mapped_column(primary_key=True)
085     occurred_at: Mapped[datetime] = mapped_column(
086         DateTime(timezone=True), default=utc_now, index=True
087     )
088     actor: Mapped[str] = mapped_column(String(80), index=True)
089     action: Mapped[str] = mapped_column(String(80), index=True)
090     outcome: Mapped[str] = mapped_column(String(24), index=True)
091     source_ip: Mapped[str | None] = mapped_column(String(64), nullable=True)
092     correlation_id: Mapped[str] = mapped_column(String(64), index=True)
093     details: Mapped[dict] = mapped_column(JSON, default=dict)
094
095
096 class CallSession(Base):
097     __tablename__ = "call_sessions"
098
099     id: Mapped[int] = mapped_column(primary_key=True)
100     session_id: Mapped[str] = mapped_column(String(128), unique=True, index=True)
101     actor: Mapped[str] = mapped_column(String(80), index=True)
102     source_extension: Mapped[str] = mapped_column(String(16), index=True)
103     destination: Mapped[str] = mapped_column(String(16), index=True)
104     media: Mapped[str] = mapped_column(String(12), default="audio")
105     state: Mapped[str] = mapped_column(String(24), default="started", index=True)
106     started_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utc_now, index=True)
107     answered_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
108     ended_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
109     hcid: Mapped[bool] = mapped_column(Boolean, default=False)
110     transferred_to: Mapped[str | None] = mapped_column(String(16), nullable=True)
111     packets_received: Mapped[int] = mapped_column(Integer, default=0)
112     packets_lost: Mapped[int] = mapped_column(Integer, default=0)
113     jitter_ms: Mapped[float | None] = mapped_column(Float, nullable=True)
114     rtt_ms: Mapped[float | None] = mapped_column(Float, nullable=True)
115     bitrate_kbps: Mapped[float | None] = mapped_column(Float, nullable=True)
```

Evidencia del modelo ORM: usuarios, presencia, extensiones, CDR, auditoría, sesiones y evaluaciones.

main.py: inicialización real de FastAPI

```
001 import asyncio
002 from contextlib import asynccontextmanager, suppress
003
004 from fastapi import FastAPI
005 from fastapi.middleware.cors import CORSMiddleware
006 from sqlalchemy import select
007
008 from app.config import get_settings
009 from app.database import SessionLocal, engine
010 from app.models import Base, Extension, User
011 from app.realtime import run_ami_listener
012 from app.routers import auth, calls, events, governance
013 from app.security import encrypt_secret, hash_password
014
015
016 async def seed_demo_users() -> None:
017     settings = get_settings()
018     seeds = [
019         ("agente1", "Agente 2001", "AgenteCallCenter", "2001", settings.demo_agent_password, settings.webrtc_2001_secret),
020         ("agente2", "Agente 2002", "AgenteCallCenter", "2002", settings.demo_agent2_password, settings.webrtc_2002_secret),
021         ("supervisor", "Supervisor QA", "Supervisor", None, settings.demo_supervisor_password, None),
022         ("adminqa", "Administrador QA", "AdministradorQA", None, settings.demo_admin_password, None),
023     ]
024     async with SessionLocal() as db:
025         for username, display_name, role, number, password, sip_secret in seeds:
026             user = await db.scalar(select(User).where(User.username == username))
027             if user is None:
028                 user = User(
029                     username=username,
030                     display_name=display_name,
031                     password_hash=hash_password(password),
032                     role=role,
033                     active=True,
034                 )
035                 db.add(user)
036                 await db.flush()
037             if number and sip_secret:
038                 extension = await db.scalar(select(Extension).where(Extension.number == number))
039                 if extension is None:
040                     db.add(
041                         Extension(
042                             number=number,
043                             user_id=user.id,
044                             sip_secret_encrypted=encrypt_secret(sip_secret),
045                         )
046                     )
047             await db.commit()
048
049
050 @asynccontextmanager
051 async def lifespan(_: FastAPI):
052     async with engine.begin() as connection:
053         await connection.run_sync(Base.metadata.create_all)
054     await seed_demo_users()
055     ami_task = asyncio.create_task(run_ami_listener())
056     try:
057         yield
058     finally:
059         ami_task.cancel()
060         with suppress(asyncio.CancelledError):
061             await ami_task
062         await engine.dispose()
063
064
065 app = FastAPI(
066     title="Unified Telecom QA API",
067     version="1.2.0",
068     description="Usuarios, RBAC, CDR, auditoria, metricas y aprovisionamiento.",
069     lifespan=lifespan,
070 )
071 app.add_middleware(
072     CORSMiddleware,
073     allow_origins=["https://localhost", "https://localhost:8443"],
074     allow_credentials=False,
075     allow_methods=["GET", "POST", "PUT", "PATCH", "OPTIONS"],
076     allow_headers=["Authorization", "Content-Type", "X-Provisioning-Token"],
077 )
078 app.include_router(auth.router)
079 app.include_router(calls.router)
080 app.include_router(governance.router)
081 app.include_router(events.router)
082
083
084 @app.get("/health", tags=["operations"])
085 async def health():
086     return {"status": "ok", "service": "backend"}
```

Evidencia de arranque: creación de tablas, seed de usuarios demo y AMI listener.

15. Evaluación frente a un call center real

Para una presentación ante jurado técnico o visitantes de un entorno de call center, el proyecto debe explicarse correctamente. La plataforma cubre comunicaciones, monitoreo, calidad, auditoría, reportes y control de usuarios, dentro de un alcance académico y de laboratorio.

Capacidad esperada en call center	Estado en el ZIP	Recomendación
Telefonía de agentes	Implementado con SIP/WebRTC.	Mostrar llamada 1001-1002 o 2001-2002.
Historial de llamadas CDR	Implementado.	Mostrar filtros/paginación y registro ANSWERED.
Grabaciones	Implementado según QA.	Mostrar descarga protegida y aclarar retención de laboratorio.
Monitoreo supervisor	Implementado.	Mostrar active calls y métricas.
Evaluación QA	Implementado.	Mostrar creación/consulta de evaluación.

16. Resiliencia de base de datos: fase pendiente prioritaria

El sistema actual usa una única instancia PostgreSQL. Esto es aceptable para laboratorio, pero no para continuidad de negocio. Si PostgreSQL cae, el backend no podría consultar usuarios, CDR, auditoría, reportes ni evaluaciones. Docker puede reiniciar el contenedor, pero reinicio no equivale a alta disponibilidad.

Elemento	Estado actual	Mejora propuesta
PostgreSQL	Una instancia principal con volumen persistente.	PostgreSQL primario + réplica streaming.
Failover	No existe failover automático.	Patroni o solución equivalente con elección de líder.
Conexiones API	Backend apunta a una base.	PgBouncer/HAProxy para redirigir al nodo activo.
Backups	No se evidencia política productiva completa.	Backup automático, prueba de restauración y retención.
Pruebas de recuperación	Fuera de alcance inicial.	Prueba de caída de primario y lectura en réplica.

17. Cronograma y Gantt recomendado

El roadmap original está expresado por fases verificables. Para entregar un avance tipo final al 90%, se presenta un cronograma con fechas absolutas desde el 09/06/2026 hasta la validación técnica del 22/06/2026, dejando únicamente como fase pendiente la resiliencia de base de datos PostgreSQL.

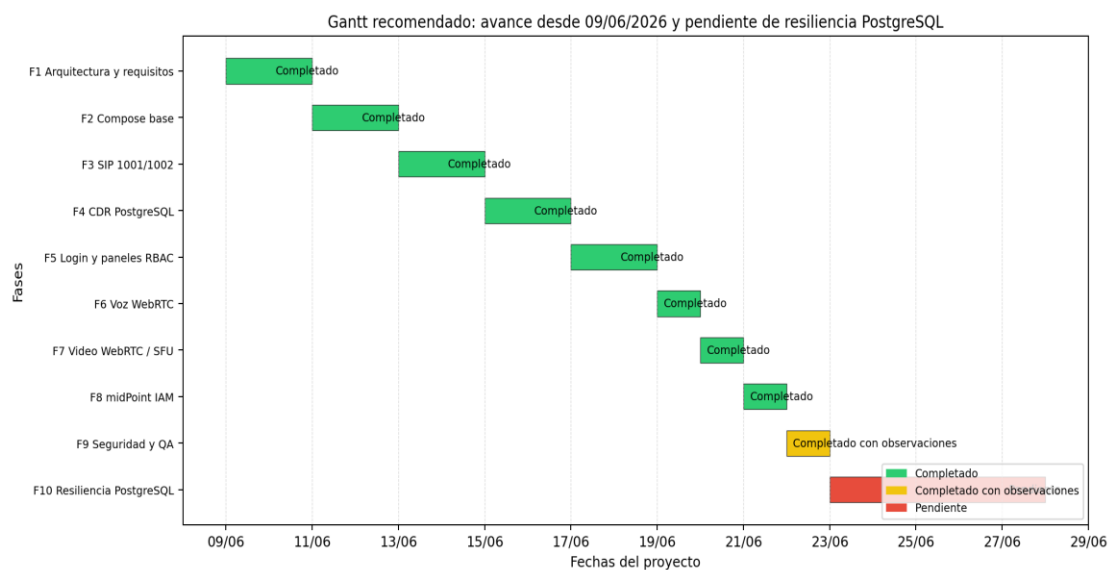


Figura 6. Gantt recomendado: avance desde el 09/06/2026 y fase pendiente de resiliencia PostgreSQL.

Fase	Inicio	Fin	Estado
F1 Arquitectura y requisitos	2026-06-09	2026-06-10	Completado
F2 Compose base	2026-06-11	2026-06-12	Completado
F3 SIP 1001/1002	2026-06-13	2026-06-14	Completado
F4 CDR PostgreSQL	2026-06-15	2026-06-16	Completado
F5 Login y paneles RBAC	2026-06-17	2026-06-18	Completado
F6 Voz WebRTC	2026-06-19	2026-06-19	Completado
F7 Video WebRTC / SFU	2026-06-20	2026-06-20	Completado
F8 midPoint IAM	2026-06-21	2026-06-21	Completado
F9 Seguridad y QA	2026-06-22	2026-06-22	Completado con observaciones
F10 Resiliencia PostgreSQL	2026-06-23	2026-06-27	Pendiente

18. Riesgos, limitaciones y mitigación

ID	Riesgo	Prob.	Impacto	Mitigación
R-01	Caída de PostgreSQL	Alta	Alta	No existe réplica; implementar resiliencia BD.
R-02	Certificados autofirmados	Media	Media	Usar PKI corporativa en producción.
R-03	Exposición de secretos .env	Media	Alta	No capturar ni versionar .env; usar secrets manager.
R-04	Compatibilidad de softphones físicos	Media	Media	Probar Zoiper/Linphone/MicroSIP en la red real.
R-05	Grabaciones con datos personales	Media	Alta	Cifrado, consentimiento, retención y borrado formal.
R-06	Dependencia de un solo host	Media	Alta	Plan multinodo y monitoreo externo.

19. Estado de avance y brechas controladas

Componente	Estado	Avance estimado
------------	--------	-----------------

Arquitectura y documentación	Completado	100%
Docker Compose e infraestructura base	Completado	100%
Telefonía SIP tradicional	Completado	100%
WebRTC voz/video	Completado con observaciones	95%
Backend API y RBAC	Completado	100%
Base de datos operativa	Completado sin HA	85%
Calidad y seguridad	Completado con observaciones	90%
Resiliencia PostgreSQL	Pendiente	0%

Promedio interpretativo: el nucleo tecnologico es demostrable y se encuentra preparado para sustentacion academica. Las brechas restantes no invalidan la demostracion, pero deben declararse con honestidad tecnica: alta disponibilidad productiva, integracion PSTN real, endurecimiento PKI corporativo y operacion multinodo quedan como mejoras futuras.

20. Conclusiones

- La plataforma demuestra una integración técnica sólida entre Asterisk, SIP.js, React, FastAPI, PostgreSQL, midPoint, coturn y Nginx, logrando una arquitectura modular orientada a comunicaciones unificadas, monitoreo y aseguramiento de calidad.
- El sistema es adecuado como plataforma académica de comunicaciones unificadas para entornos de call center, permitiendo autenticación por roles, telefonía SIP/WebRTC, gestión de usuarios, monitoreo de llamadas, auditoría, reportes y evaluación de calidad.
- La arquitectura implementada evidencia la aplicación práctica de conceptos de ingeniería de software, redes, seguridad, bases de datos, virtualización y aseguramiento de calidad, integrando múltiples tecnologías dentro de un entorno dockerizado y reproducible.
- Los resultados obtenidos durante las pruebas funcionales y de calidad demuestran un comportamiento estable de los componentes principales del sistema, validando la operación de los servicios de telefonía, backend, base de datos, autenticación, monitoreo y generación de reportes.
- El proyecto constituye una base tecnológica sólida para futuras extensiones orientadas a escenarios empresariales, manteniendo una separación clara entre los componentes de presentación, lógica de negocio, telefonía, persistencia de datos y gestión de identidades.

21. Trabajos futuros

- Implementar un esquema de resiliencia para PostgreSQL mediante replicación y mecanismos de failover que permitan mejorar la disponibilidad del servicio ante fallos.
- Incorporar procedimientos formales de respaldo, restauración y recuperación ante desastres para fortalecer la continuidad operativa de la plataforma.
- Realizar pruebas de validación con dispositivos SIP físicos y escenarios de comunicación más cercanos a entornos organizacionales reales.
- Ampliar las capacidades de monitoreo y observabilidad mediante métricas centralizadas, alertas y tableros de supervisión en tiempo real.

- Evaluar estrategias de despliegue multinodo y automatización de infraestructura que faciliten la evolución de la plataforma hacia entornos de mayor escala.

Anexo A. Archivos reales analizados

- README.md
- docker-compose.yml
- backend/app/main.py
- backend/app/models.py
- backend/app/routers/auth.py
- backend/app/routers/calls.py
- backend/app/routers/events.py
- backend/app/routers/governance.py
- frontend/src/sip.ts
- infrastructure/asterisk/configs/extensions.conf
- docs/qa_report.md
- docs/evidence/final-validation.md
- docs/planning/roadmap.md
- docs/ports.md
- docs/requirements/srs.md
- docs/security/threat-model.md

Anexo B. Puertos declarados

Puerto	Protocolo	Servicio	Uso
80	TCP/HTTP	Nginx	Redirección a HTTPS
443	TCP/HTTPS/WSS	Nginx	Frontend, API y SIP WebSocket
5060	UDP/SIP	Asterisk	Softphones tradicionales, solo laboratorio
5061	TCP/SIP TLS	Asterisk	Softphones protegidos
10000-10100	UDP/RTP/RTCP	Asterisk	Medios de llamada
3478	UDP/TCP STUN/TURN	coturn	Descubrimiento y relay
5349	UDP/TCP TURN TLS	coturn	Relay protegido
49160-49200	UDP	coturn	Puertos de relay
8080	TCP/HTTP	midPoint	Administración IAM local
9000	TCP/HTTP	SonarQube	Perfil QA opcional
5038	TCP/AMI interno	Asterisk	Eventos hacia FastAPI; nunca publicado al host

Anexo C. Resumen de roadmap original

Fase	Entregable verificable	Criterio de salida	Evidencia mínima
1	Arquitectura, requisitos, historias, matrices y repo	Documentos coherentes y enlaces válidos	EV-F1-01 inventario; EV-F1-02 árbol; EV-F1-03 revisión
2	Compose base	Servicios arrancan con health checks o limitación explícita	docker compose ps, health endpoints
3	PJSIP 1001/1002	Ambos registran y completan llamada SIP	CLI PJSIP y captura de llamada
4	CDR PostgreSQL	Llamada genera fila correcta y consultable	Consulta SQL anonimizada
5	Login y paneles	Rutas y API aplican los tres roles	Capturas y matriz 200/403
6	Voz WebRTC	Registro WSS y audio bidireccional	DevTools, CLI y CDR

7	Video WebRTC	Video VP8 compatible o degradacion a audio	webrtc-internals redactado
8	midPoint	Alta/cambio/baja idempotente	Tarea IAM, API y estado Asterisk
9	Seguridad	TLS/WSS/SRTP, redes y logs seguros verificados	Protocolos, puertos y pruebas negativas
10	QA final	Suites y analizadores ejecutados; riesgos declarados	Reportes, matriz final y video

22. Correcciones incorporadas por auditoria senior QA

Esta seccion consolida las mejoras solicitadas despues de la revision tecnica del documento. Su objetivo es cerrar brechas de sustentacion, explicar componentes que podian quedar ambiguos y conectar el proyecto con Programacion de Interfaces y Dispositivos Perifericos, Calidad de Software, seguridad e infraestructura.

22.1 Integracion con dispositivos perifericos

El proyecto si utiliza perifericos, aunque no son dispositivos externos industriales. En este laboratorio los perifericos principales son microfono, camara, parlantes o auriculares y adaptador de red. El navegador accede a ellos mediante la API MediaDevices de WebRTC y los usa para capturar audio y video en llamadas SIP sobre WebSocket seguro.

Periferico	Uso en el proyecto	Control de calidad o seguridad
Microfono	Captura audio del agente para llamadas WebRTC y softphone.	Permiso explicito del navegador/app; prueba funcional de audio.
Camara	Captura video local para videollamada.	Permiso explicito; validacion de vista local y negociacion SDP.
Parlante/Audifono	Reproduce audio remoto de la llamada.	Prueba de llamada respondida y verificacion manual de audio.
Interfaz de red	Transporta SIP, WebSocket, RTP/SRTP y trafico HTTPS.	Puertos documentados; separacion por redes Docker.
Celular iPhone/Android	Actua como endpoint SIP tradicional mediante Linphone.	Registro SIP, llamada 1001-1002 y evidencia CDR.

22.2 Configuracion de softphones moviles

Para la sustentacion se recomienda probar con Linphone porque esta disponible en iPhone y Android y permite usar cuentas SIP de terceros. El telefono debe estar en la misma red Wi-Fi que la PC, salvo que se configure NAT, VPN o publicacion controlada de puertos.

Dispositivo	Usuario	Clave demo	Dominio / servidor	Transporte
iPhone con Linphone	1001	Sip1001!	IP_DE_LA_PC:15060	UDP
Android con Linphone	1002	Sip1002!	IP_DE_LA_PC:15060	UDP
WebRTC navegador	agente1 / extension 2001	Demo1234! en portal	https://localhost:18443	WSS + DTLS-SRTP

Si el profesor solicita probar en su celular, se debe reemplazar IP_DE_LA_PC por la IP Wi-Fi vigente de la computadora donde corre Docker. Si cambia la red, la IP puede cambiar y se debe reconfigurar el laboratorio movil.

22.3 Puertos corregidos: host, contenedor y funcion

La documentacion anterior listaba puertos conceptuales. Para evitar confusion ante jurado, se diferencia entre puerto interno del contenedor y puerto expuesto en Windows/Docker Desktop.

Uso	Puerto host	Puerto contenedor	Protocolo	Servicio
Frontend/API HTTPS	18443	8443	TCP	nginx
Frontend/API HTTP laboratorio	18080	8080	TCP	nginx
SIP tradicional UDP	15060	5060	UDP	asterisk
SIP TLS	15061	5061	TCP/TLS	asterisk
RTP audio/video	11000-11100	10000-10100	UDP	asterisk
STUN/TURN	13478	3478	UDP/TCP	coturn
TURN seguro	15349	5349	TCP/TLS	coturn
midPoint IAM	18081	8080	TCP	midpoint
SonarQube	19000	9000	TCP	sonarqube

22.4 Base de datos y persistencia

PostgreSQL se utiliza como motor de persistencia principal. No se expone directamente hacia el host por seguridad; se accede desde los contenedores de backend y midPoint o mediante docker compose exec. Esto reduce superficie de ataque y mantiene la base dentro de la red interna del laboratorio.

Base de datos	Usuario tecnico	Proposito	Tablas clave
telecom_qa	telecom_app	Aplicacion principal de telecomunicaciones.	users, extensions, call_detail_records, audit_events, call_sessions, quality_evaluations
midpoint	midpoint	Repositorio interno de identidad midPoint.	m_object, m_assignment, m_audit_event y tablas propias de midPoint

La evidencia minima de base de datos debe mostrar: usuarios demo, extensiones asignadas, CDR con estado ANSWERED/FAILED y eventos de auditoria de login o llamadas.

22.5 ISO/IEC 9126 y NTP 9126

Ademas de ISO/IEC 25010, se incorpora ISO/IEC 9126/NTP 9126 porque sigue siendo comun en evaluacion academica de calidad. La relacion entre ambos modelos permite explicar que el proyecto evalua funcionalidad, confiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad.

Caracteristica ISO/IEC 9126 / NTP 9126	Aplicacion en el proyecto	Evidencia sugerida
Funcionalidad	Login, roles, llamadas, CDR, reportes y auditoria.	Demo de agente, supervisor y QA.
Confiabilidad	Healthchecks, smoke tests y reinicio de contenedores.	docker compose ps y smoke_test.
Usabilidad	Panel de agente con estado, linea de salida, historial y acciones claras.	Capturas de frontend.
Eficiencia	Pruebas k6 y respuesta de API.	Reporte de carga.
Mantenibilidad	Microservicios, documentacion, pruebas y separacion de configuracion.	Estructura del repositorio y SonarQube.
Portabilidad	Docker Compose y variables .env.	Levantamiento reproducible en Docker Desktop.

22.6 CMMI: gestion y aseguramiento del proceso

CMMI no se demuestra solo con codigo: se evidencia con disciplina de ingenieria. Por eso se vinculan requisitos, validacion, verificacion, configuracion y aseguramiento de calidad.

Area CMMI	Como se evidencia	Artefacto recomendado
Gestion de requisitos	Requisitos funcionales/no funcionales y trazabilidad.	SRS, user stories y matriz RF.
Verificacion	Pruebas unitarias, integracion, humo, e2e y calidad estatica.	pytest, vitest, Playwright, SonarQube.
Validacion	Demostracion de llamadas reales y flujo de usuario.	Video de demo y checklist de evidencia.
Aseguramiento de calidad	Reporte QA, ISO 25010/9126 y gates de calidad.	qa_report.md y resultados de pruebas.
Gestion de configuracion	Repositorio Git, .env.example, Docker Compose y commits.	Historial Git y README.

22.7 Herramientas QA: que son y para que sirven

Para que un estudiante principiante pueda defender el proyecto, se aclara que estas herramientas no son lenguajes de programacion principales sino utilidades de aseguramiento de calidad y seguridad.

Herramienta	Tipo	Para que sirve en el proyecto
pytest	Framework de pruebas Python	Ejecuta pruebas del backend FastAPI.
vitest	Framework de pruebas JavaScript/TypeScript	Ejecuta pruebas unitarias del frontend React/Vite.
Playwright	Framework de pruebas end-to-end	Automatiza navegacion real en el frontend.
k6	Herramienta de carga	Simula multiples peticiones y mide tiempos de respuesta.
SIPp	Herramienta de pruebas SIP	Permite generar llamadas SIP controladas.
Trivy	Escaner de vulnerabilidades	Analiza imagenes Docker, dependencias y secretos.
SonarQube	Plataforma de calidad de codigo	Mide bugs, vulnerabilidades, code smells y cobertura.

22.8 Correccion de alcance: laboratorio vs produccion

El documento debe evitar prometer una plataforma productiva bancaria completa. La formulacion correcta es: plataforma academica avanzada y demostrable, inspirada en un escenario financiero, con controles de seguridad y calidad alineados a estandares. Para produccion faltarian PKI corporativa, alta disponibilidad, monitoreo centralizado, backups formales, hardening de red y pruebas de recuperacion ante desastres.

22.9 Checklist de evidencia para la sustentacion

1. Ejecutar docker compose --profile qa up -d --build y capturar docker compose ps.
2. Abrir <https://localhost:18443> y evidenciar login de agente, supervisor y administrador QA.
3. Registrar WebRTC con agente1/extension 2001 y capturar estado Registered.
4. Registrar Linphone 1001/1002 en celular y hacer llamada de prueba.
5. Realizar una videollamada y mostrar permiso de camara/microfono.
6. Consultar Swagger en <https://localhost:18443/docs>.
7. Mostrar CDR y auditoria en PostgreSQL.
8. Abrir midPoint en <http://localhost:18081/midpoint> y explicar que gobierna identidades/RBAC.
9. Abrir SonarQube en <http://localhost:19000> y enseñar quality gate.
10. Guardar reporte Trivy, resultados pytest/vitest/Playwright y capturas en reports/.

22.10 Respuesta preparada para preguntas del profesor

Pregunta probable	Respuesta corta recomendada
En que sistema operativo esta montado?	El host es Windows con Docker Desktop/WSL2; los contenedores usan Linux: Debian/Alpine segun la imagen.
Por que se usa Docker?	Para reproducibilidad, aislamiento de servicios y despliegue por fases.
Que hace Asterisk?	Es la PBX: registra extensiones SIP/WebRTC, enruta llamadas y genera CDR.
Que hace coturn?	Ayuda a WebRTC a conectar audio/video cuando hay NAT o redes restrictivas.
Que hace Nginx?	Sirve como reverse proxy HTTPS/WSS y punto de entrada seguro.
Que hace midPoint?	Gestiona identidades, roles y aprovisionamiento hacia la plataforma.
Que se guarda en CDR?	Origen, destino, fecha, duracion, estado y datos tecnicos de llamada.

Esto es produccion?	No; es un laboratorio academico avanzado con controles alineados a estandares y brechas declaradas.
---------------------	---

Anexo D. Auditoria visual de imagenes y evidencias

Se revisaron las 13 imagenes embebidas en el documento corregido. Todas conservan integridad interna dentro del archivo DOCX y cumplen una funcion de evidencia academica. Las capturas de codigo y configuracion son validas como soporte tecnico; durante la sustentacion se recomienda ampliarlas en pantalla si el jurado desea revisar lineas especificas.

Grupo visual	Resultado QA	Observacion
Logo UTP	Apto	Identificacion institucional en portada.
Diagramas de arquitectura/red/casos de uso/ER	Apto	Sirven para explicar componentes, relaciones y flujo general.
Capturas de codigo y configuracion	Apto con recomendacion	Funcionan como evidencia, pero deben ampliarse si se explican lineas exactas.
Gantt y roadmap	Apto	Permite sustentar avance, fases y brechas pendientes.
Capturas de Asterisk/extensions	Apto	Evidencian configuracion real de telefonia SIP/WebRTC.

Conclusion de auditoria visual: no se requiere reemplazar imagenes para la entrega actual. La mejora aplicada fue documentar su funcion probatoria y anadir descripciones alternativas para accesibilidad basica.

Anexo E. Sustentacion orientada al proceso de calidad

El profesor recalco que el criterio principal no es solamente que la plataforma funcione en una demostracion, sino que exista un proceso de calidad defendible. Por ello, la sustentacion debe demostrar que el equipo siguio un ciclo ordenado: planificar, implementar, verificar, validar, registrar evidencias, analizar riesgos y proponer mejoras.

E.1 Idea central para defender el proyecto

La frase guia de la exposicion es: No construimos una aplicacion al azar; seguimos un proceso de ingenieria de software y aseguramiento de calidad. La plataforma es la evidencia tecnica, pero el valor academico principal esta en los requisitos, la arquitectura, las pruebas, la trazabilidad, los controles y la mejora continua.

E.2 Proceso aplicado por fases

Fase del proceso	Que se hizo	Evidencia que se muestra	Relacion con calidad
Requisitos	Se definieron funcionalidades, roles, restricciones y criterios de aceptacion.	requirements.md, user stories, matriz RF.	Gestion de requisitos y trazabilidad CMMI.
Arquitectura	Se separaron servicios: frontend, backend, PBX, base de datos, IAM, proxy y QA.	architecture.md, diagramas y Docker Compose.	Mantenibilidad, portabilidad y seguridad ISO 25010.
Implementacion incremental	Se trabajo por fases: Docker, Asterisk, CDR, WebRTC, midPoint, seguridad y QA.	Commits, estructura del repo y roadmap.	Gestion de configuracion y control de cambios.
Verificacion	Se ejecutaron pruebas unitarias, integracion, humo, e2e, carga y seguridad.	pytest, vitest, Playwright, k6, Trivy y SonarQube.	Verificacion CMMI e ISO/IEC 9126.
Validacion	Se probaron flujos reales: login, llamadas, CDR, auditoria, reportes y roles.	Capturas, video demo, Swagger y CDR en BD.	Validacion con usuario y adecuacion funcional.
Gestion de riesgos	Se declararon limites: certificados, red, NAT, video, alta disponibilidad y produccion.	risk_matrix.md y seccion de brechas.	Prevencion, mejora continua e ISO 27001.
Mejora continua	Se corrigieron incidencias como puertos, Swagger, documentacion e imagenes.	Historial Git, QA report y version final del Word.	Ciclo PDCA: planificar, hacer, verificar, actuar.

E.3 Como responder si algo falla en la demo

Si durante la exposicion falla un servicio, una llamada o una pagina, no debe interpretarse como fracaso del proyecto. En calidad de software, una falla controlada se convierte en evidencia de proceso si el equipo sabe diagnosticarla, registrarla y proponer una accion correctiva.

Situacion	Respuesta correcta ante el profesor	Evidencia o accion
Docker no levanta	Se revisan contenedores, logs, puertos y healthchecks.	docker compose ps y docker compose logs.
Swagger no abre	Se verifica Nginx, CSP, /docs y /openapi.json.	Header CSP corregido y prueba 200 OK.
Llamada no conecta	Se revisa registro SIP, red Wi-Fi, NAT, codecs y puertos RTP.	pjsip show contacts, CDR y logs Asterisk.
No hay video	Se valida permiso de camara, codec, SDP y compatibilidad del softphone.	Captura WebRTC/Linphone y seccion de riesgos.

Base de datos no responde	Se diagnostica PostgreSQL, volumen, credenciales y healthcheck.	psql, tablas users/CDR/audit_events.
midPoint demora	Se explica que IAM tiene arranque pesado y depende de PostgreSQL.	Healthcheck y acceso a consola.

E.4 Guion breve de sustentacion centrado en proceso

Paso 1: Problema: un call center necesita comunicaciones, roles, trazabilidad, CDR, auditoria y metricas de calidad.

Paso 2: Requisitos: se definieron funcionalidades, restricciones, roles y criterios de aceptacion antes de implementar.

Paso 3: Arquitectura: se eligio microservicios con Docker para separar responsabilidades y mejorar portabilidad.

Paso 4: Implementacion: se avanza por fases controladas, no como un unico cambio grande.

Paso 5: Verificacion: se ejecutaron pruebas automaticas, analisis estatico, analisis de vulnerabilidades y smoke tests.

Paso 6: Validacion: se demostraron flujos reales con agentes, supervisor, administrador QA, softphones y navegador.

Paso 7: Riesgos: se declararon limites reales del laboratorio y diferencias frente a produccion.

Paso 8: Mejora continua: cada hallazgo se transformo en correccion, documento, commit o accion futura.

E.5 Relacion con estandares

Estandar o modelo	Como se evidencia en el proceso
ISO/IEC 25010	Se evalua funcionalidad, seguridad, mantenibilidad, portabilidad, compatibilidad y eficiencia.
ISO/IEC 9126 / NTP 9126	Se refuerzan funcionalidad, confiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad.
ISO/IEC 27001	Se consideran control de acceso, proteccion de credenciales, trazabilidad, red segura y vulnerabilidades.
CMMI	Se evidencia gestion de requisitos, verificacion, validacion, aseguramiento de calidad y configuracion.
PDCA	Se planifica, se implementa, se verifica con pruebas y se actua corrigiendo hallazgos.

E.6 Conclusion para la defensa

La plataforma puede demostrarse funcionando, pero la defensa academica debe destacar el proceso. Incluso si una parte de la demo falla, el equipo puede explicar el diagnostico, el riesgo, la evidencia y la mejora propuesta. Ese enfoque demuestra madurez de calidad de software y es coherente con CMMI, ISO 25010, ISO 9126/NTP 9126 e ISO 27001.